A Two-Tier Biology-Aware Wake-Up Control for Energy-Measured IoT Insect Monitoring

Francesco Betti Sorbelli, *Member, IEEE*, Enrico Casella, *Member, IEEE*, Lorenzo Palazzetti, *Member, IEEE*, and Cristina M. Pinotti, *Senior Member, IEEE*

***Abstract*—Agricultural pest-monitoring Internet of Things (IoT) systems face a dilemma: continuous image capture drains batteries within weeks while storing mostly irrelevant frames. This paper targets the acquisition decision itself. We present a two-tier monitoring system in which a low-power ESP32-class Microcontroller Unit (MCU) decides when a Raspberry Pi (RPi) camera node is worth powering on, so that expensive perception runs only under relevant conditions. The controller executes two lightweight on-device Machine Learning (ML) models: a quantized recurrent *forecaster* that predicts near-future weather from a single reading, and a two-stage, biologically informed wake-up *decider* that gates capture using weather and insect-phenology cues. The decider combines a deterministic *biological envelope*, based on calendar, temperature, and degree-day thresholds, with a learned *refinement*. We evaluate it on a multi-station dataset of Brown Marmorated Stink Bug (BMSB) activity. The biological envelope alone removes 72.9% of the captures in an annual blanket-capture schedule (one image every 5 minutes) while retaining 99.9% of relevant detections, and the full decider achieves a more favorable capture-reduction–activity-retention trade-off than the blind random-check baselines under the conservative cross-camera protocol. Measured on real hardware, the controller itself adds at most 0.055% overhead relative to the perception tier it gates. Per-inference energy confirms that on-device forecasting is negligible next to a single image capture. Forecasting is $4.06\times$ cheaper at the shortest, most reliable 1-hour prediction horizon, and, once the perception tier's idle-hold power is properly accounted for, the deployed HistGBT decider reduces modeled annual RPi-side energy by a factor of about 181 relative to a continuously idle-held blanket-capture policy.**

***Index Terms*—Internet of Things, Edge Computing, Energy Efficiency, Machine Learning, Brown Marmorated Stink Bug**

I. Introduction

Battery-powered agricultural Internet of Things (IoT) nodes are increasingly used to monitor pest activity in the field [1], but image-based monitoring remains expensive: every capture wakes hardware, stores data, and may trigger transmission even when the target species is unlikely to be active [2]. For a grower, this creates a familiar tradeoff between labor-intensive field scouting and automated camera-based monitoring [3]. Neither option scales. For seasonal pests, the core problem is therefore scheduling as much as perception: the system must decide not only *what* is in an image, but also *when* acquiring an image is worth the energy cost.

We study this problem through the Brown Marmorated Stink Bug (BMSB) (*Halyomorpha halys* (HH)), a concrete and economically important case. First reported in Italy in 2012 [4], the BMSB has since become invasive across Northern Italy and other European countries, threatening crops from pears to hazelnuts and disrupting the Integrated Pest Management (IPM) programs that growers rely on to avoid blanket spraying. Since the species is common, seasonal, and locally clustered where it has established, the monitoring problem is not whether it can be detected, but whether it can be observed continuously, in the field, within the energy budget of a practical deployment. The same acquisition problem extends to other seasonal pests whose activity is shaped by weather, phenology, and local aggregation.

This acquisition cost exposes a blind spot in current IoT monitoring. Automated field cameras can collect large volumes of data [1], but their acquisition schedules usually answer *when to look* in the same way: at fixed intervals [1], [2], [5], regardless of whether target activity is plausible. The result is not only wasted energy but wasted attention: many captures occur in hours when the target is biologically unlikely to be active, yet each still costs a wake, a capture, and a transmission. Work that specifically targets embedded efficiency reduces model complexity [6] or offloads computation to the cloud [7], and work that targets coverage reaches for higher-performance hardware [8] or drone-based surveys [9], [10], [11]; however, in each case, *when* to sense remains fixed or continuous rather than conditioned on the biology of the insect being sensed. The missing decision is simple and operational: *is this moment worth looking at?*

This paper answers that question on-device, with a two-tier system that separates *decision* from *perception*. A low-power ESP32-class Microcontroller Unit (MCU) stays in deep sleep and briefly wakes to run two lightweight models, a quantized Gated Recurrent Units (GRU) *weather forecaster* and a biologically informed *wake-up decider*. The MCU also gates the *perception tier*, that is, it powers a Raspberry Pi (RPi) camera node only when BMSB activity is likely. The decider reframes acquisition as a rare-event detection problem, combining weather variables (temperature, humidity) with insect-phenology indicators (degree-day accumulation, generational thresholds). The forecaster lets the controller look

Manuscript received Month xx, 2026; revised Month xx, 2026; accepted Month xx, 2026. Date of publication Month xx, 2026; date of current version Month xx, 2026. (*Corresponding author: Francesco Betti Sorbelli.*)

This work was partially supported by the "GNCS – INdAM". F. Betti Sorbelli, L. Palazzetti, and C. M. Pinotti are with the GNCS – INdAM. F. Betti Sorbelli and C. M. Pinotti are with the Dept. of Computer Sci. and Mathematics, Univ. of Perugia, Italy; E. Casella and L. Palazzetti are with the Dept. of Animal Sci., Pennsylvania State Univ., USA.

This work was supported by Penn State Inst. for Computational and Data Sci. (RRID:SCR_025154) providing access to computational research infrastructure (RRID:SCR_026424), the USDA National Inst. of Food and Agriculture and Hatch Appropriations under Project #PEN05072 and Accession #7009799. Digital Object Identifier XXXXXXXX/ZZZ.YYYYYYYY

ahead without spending an extra sensor wake-up for every step of that look-ahead. We evaluate this design on a multi-station dataset [12] under a cross-station protocol (train on one camera, score another), and measure controller energy directly on the target hardware with an inline power meter.

The measured payoff is concrete under the stated evaluation and energy-model assumptions. A deterministic *biological envelope* (threshold checks against calendar, clock, and temperature, with no learned model) removes 72.9% of the captures in an annual 5-minute schedule, while correctly catching 99.9% of real activity. Moreover, the decider achieves a more favorable capture-reduction–activity-retention trade-off than blind random-check policies, which capture each bin independently with fixed probability p, under the cross-camera protocol that a newly deployed station actually faces. The weather forecaster feeding this decider degrades gracefully as the look-ahead grows and quantizes to `int8` at no accuracy cost. On real hardware, a complete controller wake for sensing, forecasting, and decision-making costs at most 0.055% of one perception-tier wake. The forecast-driven schedule cuts the energy spent on weather anticipation by $4.06\times$ compared with continuous sensing at the shortest, most reliable 1-hour horizon; once the perception tier's idle-hold power is properly accounted for, the deployed HistGBT wake-up policy reduces modeled annual RPi-side energy by a factor of 181 relative to a continuously idle-held blanket-capture policy.

This article extends our preliminary conference paper [13], which introduced the Machine Learning (ML) wake-up mechanism for insect detection on an initial dataset. The contributions of our paper are as follows:

- We propose a two-tier design: a low-power ESP32 gates an on-demand RPi perception node, assigning forecasting, wake-up, and imaging to the lowest-energy device.
- We design two lightweight on-controller models: a quantized GRU weather forecaster deployed through a hand-written kernel, and a two-stage, biologically informed wake-up decider built from a deterministic biological envelope and a learned refinement.
- We characterize controller energy *directly on the target hardware*, including per-phase currents, per-inference energy, quantization cost, and a composed forecast-sense comparison; we incorporate perception-tier costs in a separate system-level projection.
- We validate the wake-up decider on a multi-station BMSB dataset under a deployment-oriented cross-station protocol, rather than only under same-camera evaluation.

The paper is organized as follows: Section II surveys relevant work. Section III presents the two-tier architecture and its two on-controller models. Section IV describes the dataset behind the wake-up decider. Section V evaluates the forecaster, the wake-up decider, and the measured energy budget in turn. Section VI discusses the implications and limitations of these results, and Section VII concludes.

II. Related Work

This section positions the work across energy-efficient edge sensing, automated insect monitoring, and embedded forecasting on MCUs, identifying the gap at their intersection.

A. Energy-Efficient Edge IoT and Wake-on-Event Sensing

Extending the operational lifetime of battery-powered IoT nodes is a long-studied problem. Energy-management techniques for "Green-IoT" are surveyed in [14], and duty-cycling (keeping a node asleep except for brief, scheduled or triggered activity windows) remains the dominant strategy [15]. For nodes that harvest ambient energy rather than relying solely on a battery, prior work reviews how the duty cycle must adapt to the incoming energy budget [16] and formalizes joint energy-management-and-wake-up policies that decide when a harvesting-powered node may afford to activate [17]. In both cases, however, the decision tracks the *energy* state of the node, not the state of the phenomenon it is meant to observe. Hardware wake-up radios push duty-cycling further by adding a nanowatt-class secondary receiver that rouses the main system only on an external Radio Frequency (RF) trigger [15]; such designs cut idle-listening cost, but their wake decision still depends on a fixed, low-level signal rather than on the semantics of the sensed phenomenon.

Closer to the two-tier split adopted here, SamurAI [18] combines an always-on, event-driven subsystem with an on-demand compute-intensive one, achieving a $15{,}000\times$ peak-to-idle power ratio and up to $3.5\times$ longer operation; however, its wake condition is a generic hardware event detector rather than a learned model of the target process. A parallel line of work keeps sensing continuous but shrinks the *processing* side instead: latency is reduced on constrained devices in [19], log streams are compressed with a temporal convolutional network for local inference in [20], DBSCAN is paired with PCA for lightweight intrusion detection in [21], and a graph neural network is distilled for traffic-aware anomaly detection in [22]. These approaches reduce the cost of each observation but not the number of observations, since the sensor keeps acquiring at a fixed rate. The proposed controller instead learns to skip acquisitions that are unlikely to matter, cutting active time itself rather than only the cost of each activation or the energy budget available for it.

B. Automated Insect Monitoring Hardware

Camera-based insect monitoring has moved from manual scouting to embedded pipelines, but most deployments still sample on a fixed schedule or run continuously. A detection pipeline deployed on an NVIDIA Jetson Nano [8] stores only positively classified frames, cutting storage but not the power-hungry, always-on acquisition itself. For BMSB specifically, drone or static-camera acquisition is combined with cloud-based inference in [23], [1], [5], while a drone-and-cloud monitoring system that uses historical detections to plan flight paths is developed incrementally in [11], [10], [9], [24], [7]; both lines depend on continuous connectivity and treat every pass or every frame as equally worth acquiring.

Detection is moved to the edge in [2], [6], where a smart sticky trap runs lightweight U-Net-inspired segmentation directly on the device, but acquisition still occurs at fixed intervals regardless of expected pest activity. Sensing modality varies too, not just hardware tier: Passive Infrared (PIR) motion sensing, acoustic analytics, and an ultrasonic deterrent

are combined in [25] for real-time pest response in open fields, but the system still reacts to an insect already present rather than anticipating the conditions that make its presence likely. Beyond BMSB, an edge–cloud pipeline for ant-species detection is reported in [26] with a similarly schedule-driven capture policy. Across this literature the acquisition trigger is time or the immediate physical signature of the target, never the biological or environmental state of the field ahead of time. This is the gap targeted by this work.

C. Embedded Forecasting on MCUs

A third literature asks what can run on the MCU itself. Embedded inference deployments, often grouped under the term TinyML, are surveyed in [27], [28]. These surveys show that on-device inference has become a practical alternative to cloud offloading when bandwidth, latency, or energy is constrained. They also note that recurrent and sequence models remain under-represented relative to convolutional ones. Runtime support is a concrete reason why: the dominant inference engine, TensorFlow Lite Micro [29], targets convolutional and dense layers well but does not support recurrent cells out of the box, which motivates deploying the GRU forecaster through a hand-written kernel rather than a stock runtime.

On the model side, a gated recurrent architecture is compressed to kilobyte size without sacrificing accuracy in [30], evidence that recurrent forecasting fits the memory budget of an MCU. Embedded regression of a physical quantity, rather than classification, is likewise emerging but still narrow in scope: pollutant gas concentration is predicted from low-cost sensors within the memory and compute budget of an MCU in [31], and short-term solar irradiance is forecast with a lightweight model on constrained hardware in [32]. Both demonstrate on-device time-series prediction of a physical, weather-linked quantity, but for air-quality reporting and energy-yield estimation, respectively, rather than to gate a downstream sensor. None of these works close the loop from an MCU-resident forecast to a sensing decision.

D. Positioning and Gaps

The preceding literature leaves open whether a single system can combine biological cueing, two-tier sensing, and measured energy savings. We make this gap explicit through three questions, one for each literature above.

- *Does wake-on-event sensing suffice?* Not by itself: hardware wake-up radios and harvesting-aware duty cycles [15], [16], [17] wake on a generic signal or on the energy state of the node, not on a model of the phenomenon being observed.
- *Does insect-monitoring hardware close the gap?* Not fully: drones, sticky traps, and acoustic deterrents [11], [2], [25] acquire on a timer or in reaction to the presence of the pest, rather than in anticipation.
- *Does embedded forecasting supply the missing anticipation?* Not alone: prior work [30], [31], [32] shows that a forecast can run on an MCU and regress a physical quantity, but it does not connect that forecast back to a sensing decision.

Taken together, these gaps identify three missing pieces: (i) a decision tied to the phenomenon, not to a generic trigger; (ii) anticipation rather than reaction; and (iii) a forecast that *acts*, not just reports. These missing pieces also define the extension beyond our preliminary work [13] that addressed a narrower version of this problem, showing that pest activity can be predicted from weather and phenology. It did not establish whether that prediction can be produced early and cheaply enough to gate the camera itself, nor whether the resulting energy saving holds under real hardware measurements.

This manuscript addresses these points by moving the decision onto the MCU, measuring on-target savings, and evaluating cross-station generalization.

III. System Architecture

This section describes the two hardware tiers and the control loop that connects them. The goal is simple: keep the camera node completely off unless the low-power controller predicts that an image is worth taking.

A. Two-Tier Design

The system separates *decision* from *perception*, as shown in the upper controller tier and lower perception tier of Figure 1, respectively. The *controller tier* is an ESP32-32D (ESP32) MCU. It stays in deep sleep most of the time and wakes briefly to decide whether the next time step requires a capture. During each wake, it performs three tasks: it reads or predicts the weather, combines those values with insect-phenology cues, and either keeps the camera node off or powers it through a controlled wake line. The controller therefore makes the acquisition decision, but it does not perform visual recognition.

The *perception tier* is an RPi 5 equipped with an RPi Camera Module 3. It is more capable than the ESP32, but it also consumes much more energy. For this reason, the RPi is fully off by default. It boots only after the controller authorizes a capture, takes one image, optionally filters it on-device, and then shuts down again. This split assigns continuous decision-making to the low-power tier and image acquisition to the high-power tier.

B. Controller Tier: Sensing, Forecasting, and Waking

The controller runs three mechanisms: weather tracking, wake-up scheduling, and power gating.

Weather tracking: The ESP32 reads temperature and humidity from an onboard SHT3x sensor over I^2C; a co-located BMP384 supplies pressure. Taking a physical reading at every 5-minute decision step would waste energy, because the controller must keep the bus active and then wait for the sensor conversion. Instead, the controller takes one physical reading at the beginning of a look-ahead window. A small *weather forecaster* then predicts the next steps of that window in one forward pass. The forecaster predicts all future steps directly; it avoids autoregressive approaches. We emphasize that all predictions depend on a single measurement.

The *window length* τ denotes the number of 5-minute steps and ranges from 12 to 288, corresponding to horizons

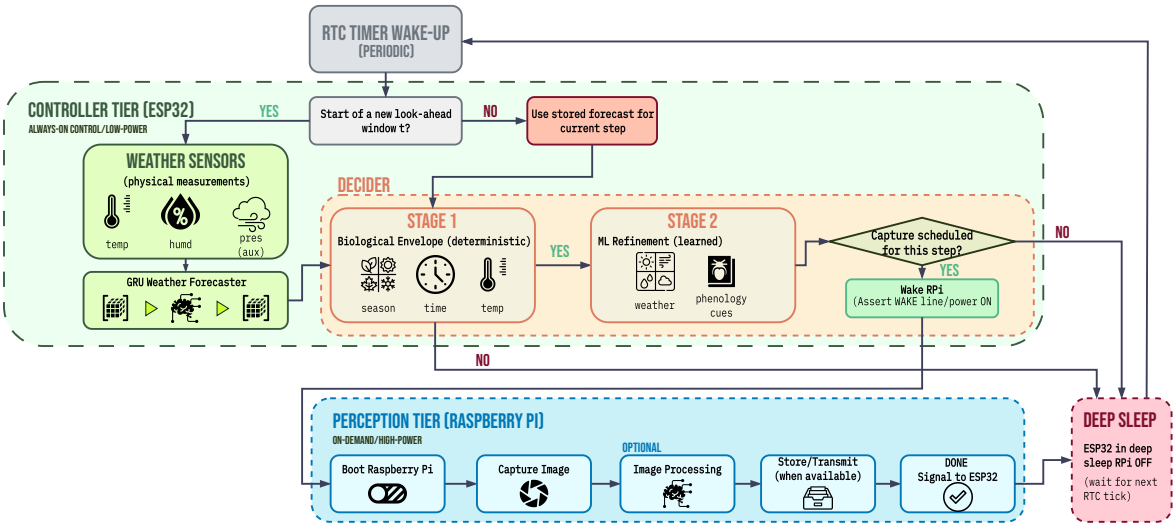


Fig. 1. Two-tier system architecture and operating flow. The upper controller tier reads or forecasts the weather, applies the wake-up decider, and powers the lower RPi 5 perception tier, equipped with an RPi Camera Module 3, only when capture is warranted.

of 1 to 24 h, respectively. A larger τ reduces the number of physical sensor reads, but also increases the time during which the system relies on predictions derived from a single measurement. The chosen τ therefore balances sensing energy against forecast drift. This mixed strategy avoids both extremes: sensing at every step and forecasting for too long without a fresh measurement.

Wake-up scheduling: The sensed and forecast weather values feed a two-stage wake-up decider. The decider returns a schedule over the current look-ahead window, not only a single yes/no value. Each schedule entry specifies whether the RPi should be powered for a capture at that time step.

Within the *decider* block of Figure 1, Stage 1 is a deterministic *biological envelope*. It considers a capture only when three low-cost conditions hold: the month is within the BMSB activity season (May–September), the local hour is within the diurnal activity window (05:00–20:00), and the predicted temperature is above the 13 °C activity threshold [33]. In the temperate European setting, relevant here, BMSB is typically bivoltine: overwintered adults become active in spring, while a second activity peak occurs in late summer or early autumn [33]. These checks remove most off-season and nighttime steps before any learned model runs, reducing the annual capture schedule by 72.9% (Section V-C). Stage 1 thus handles the clear-cut, biologically implausible steps, leaving Stage 2 to resolve the more ambiguous cases that remain plausible.

In the same *decider* block of Figure 1, Stage 2 then scores the remaining time steps with a lightweight ML model. The model uses degree-day accumulation, photoperiod-derived diapause and adult-activity cues, and short-term weather trends. Degree days are computed with the single-sine method [34] using the lower and upper BMSB developmental thresholds, 14.17 °C and 35.76 °C [35]. The photoperiod features encode seasonal diapause and adult-activity patterns [36]. The weather-trend features include temperature change, dewpoint, and vapor-pressure deficit.

Waking the perception tier: If the schedule calls for a capture at the current step, the controller asserts a digital wake line. This line drives a load switch that powers the RPi. The controller keeps the RPi powered long enough to boot, capture, and optionally filter the image. When the RPi finishes, it signals completion over the same interface. The controller then de-asserts the wake line, cuts power to the RPi, and returns to deep sleep. This handshake avoids a fixed waiting time: the perception tier stays on only until completion.

C. Perception Tier and Operating Pipeline

The lower perception tier of Figure 1 shows the operating cycle after the controller authorizes a capture. The ESP32 starts in deep sleep and wakes on a real-time-clock timer. At the beginning of a new look-ahead window, it reads the sensors, forecasts the remaining weather steps, and evaluates the wake-up decider over the entire window. The controller stores one schedule entry for each future time step. At the next τ timer events, it reuses the corresponding forecast value and schedule entry rather than running the forecaster and decider again. The schedule has two possible actions for the current step: skip, leaving the RPi off, or wake, powering the RPi.

If the current step is not scheduled for capture, the ESP32 returns to deep sleep and the RPi remains off. This is the common case, because BMSB activity is sparse. If capture is scheduled, the ESP32 powers the RPi. Once booted, the RPi captures one image. The image can be sent directly to the server, or it can first be processed by an on-device detector (e.g., YOLO or FOMO-style model). This optional detector filters *what has already been captured*; it does not decide *whether to capture*, because the ESP32 has already decided before the image exists. After capture and optional filtering, the RPi sends the completion signal, the ESP32 cuts power, and both tiers return to sleep.

Forecasting reduces the number of physical sensor reads, while the decider reduces the number of activations of the

high-power tier. Hence, the main system energy savings come from avoiding unnecessary RPi wake-ups and consequently unnecessary captures, not from filtering images after capture.

IV. Dataset and Acquisition

This section describes the data used to train and test the wake-up decider: collection, conversion of image labels to a wake/sleep target, derivation of biological indicators, and the resulting pre-modeling series.

A. Data Collection

The data combine weather records and stationary-camera images collected during the 2023 growing season in a multi-station field campaign in a pear orchard in Carpi, Italy [12]. The full dataset is publicly available. Four fixed stations recorded weather every 5 minutes, including temperature, humidity, and pressure. Each station also hosted a canopy-level camera.

Two stations provide enough positive labels for training and evaluation. We refer to them as *Camera 1* and *Camera 2*. They contain 896 and 631 positively labeled images, corresponding to 2,120 and 1,581 labeled BMSB instances, respectively. The other two stations contain only 36 and 7 positively labeled images over the same period, partly because of camera outages. These low counts mainly indicate limited imaging *coverage*; they do not prove that BMSB was absent at those sites. For this reason, we train and evaluate the wake-up decider only on *Camera 1* and *Camera 2*, instead of treating the other two stations as reliable negative series. For each selected station, analysis begins from a regular full-year 5-minute grid of 105,120 bins, retaining only instants with valid weather measurements and derived features; this excludes missing, out-of-range, or stuck sensor readings and feature warm-up periods. The resulting valid sets contain 83,573 bins for *Camera 1* and 82,166 for *Camera 2*, for a pooled total of $N = 165{,}739$ bins before applying ENV; the envelope retains 57,022 biologically plausible bins for Stage 2.

B. From Labels to a Wake/Sleep Series

Camera images were labeled with YOLO bounding boxes around individual BMSB instances. Each image label is matched by timestamp to the nearest 5-minute weather bin.

For each bin, we compute `HH_count`, the number of labeled BMSB instances observed in that bin. We then convert `HH_count` into a binary target, $\texttt{RPi_sleep} \in \{0, 1\}$. Specifically, $\texttt{RPi_sleep} = 0$ if the bin contains at least one BMSB instance (i.e., the Boolean variable `RPi_sleep` is set to `false`), and $\texttt{RPi_sleep} = 1$ otherwise (i.e., `RPi_sleep` is `true`).

Thus, a bin with $\texttt{RPi_sleep} = 0$ is a 5-minute step when the perception tier should wake, whereas a bin with $\texttt{RPi_sleep} = 1$ is a 5-minute step it can safely be skipped.

All available images were manually reviewed for BMSB presence, including those associated with time bins outside the biologically plausible activity window described by prior phenology studies [37], [36]. For each reviewed image with no BMSB we fix $\texttt{RPi_sleep} = 1$.

For time instants for which temperature and weather data are available but no images were collected, starting from January, we extend the series by assigning $\texttt{RPi_sleep} = 1$, that is, assuming the absence of BMSB according to established evidence in the literature.

C. Biological Indicator Derivation

The same temperature can have different meaning early and late in the season, and its effect also depends on day length [37]. We therefore derive biological indicators (i.e., features) from the 5-minute weather series. These include degree-day accumulation, both as a running seasonal total and as a daily increment. Degree days are computed with the single-sine method [34] and the developmental thresholds of the species [35]. We also derive photoperiod cues, including a day-length trend and a long-day/short-day flag, to represent the diapause and adult-activity windows described in [36].

Figure 2 reports the point-biserial[1] correlation [38] between each candidate feature and `RPi_sleep`. All coefficients are small ($|r| \leq 0.08$), which is expected for a rare and threshold-gated event. Nonetheless, the sine encoding of day-of-year [sin(DOY)], dewpoint, and cumulative degree-days have the highest correlations. Pressure has no useful signal ($r \approx 0$), so it is removed from the feature set.

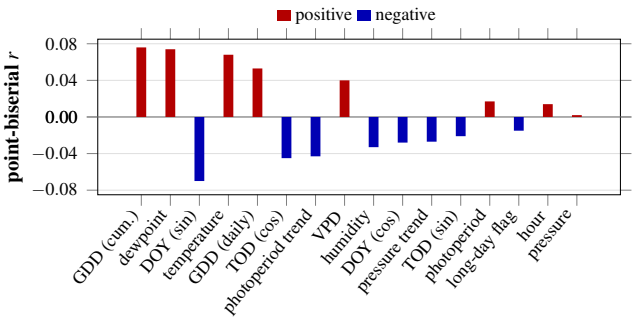


Fig. 2. Point-biserial correlation of each candidate feature with `RPi_sleep`.

D. Descriptive Overview

Figure 3 summarizes weekly BMSB activity at the two selected sites. BMSB typically has two annual activity peaks, in spring (April–May) and late summer (August–September) [33]. In this dataset, detections are concentrated in a few late-summer weeks, so the observed series is dominated by the second peak rather than representing both generations equally. *Camera 1* peaks sharply around week 37, while *Camera 2* follows a related but not identical pattern. This supports the cross-station evaluation: the two sites are similar enough to make transfer meaningful, but different enough that training on one and testing on the other is not a trivial split. Weekly counts are highly skewed, with many zero-count weeks and a few weeks with hundreds of detections. The heatmap therefore uses a log color scale so that low-count weeks remain visible.

[1] A correlation coefficient that measures the strength and direction of association between one continuous variable and one binary (dichotomous) variable.

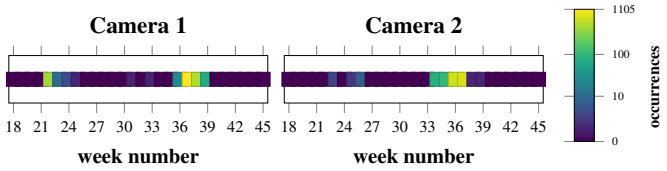


Fig. 3. Weekly BMSB occurrence counts (log scale) for *Camera 1* (left) and *Camera 2* (right).

Figure 4 compares weekly mean temperature with the average daily detection count per camera. The detection peak does not coincide with the maximum temperature. Instead, it appears after the seasonal temperature decline has started. This pattern aligns with a cumulative, degree-day-driven response rather than with a simple instantaneous temperature threshold.

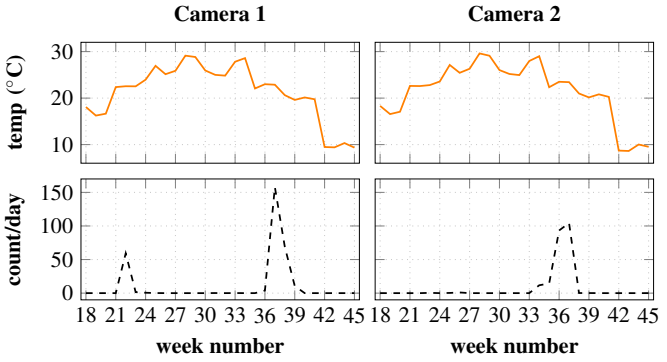


Fig. 4. Weekly mean temperature (top row) and average daily detection count (bottom row), for *Camera 1* (left column) and *Camera 2* (right column).

V. Experimental Evaluation

This section evaluates the two on-controller models and quantifies the energy implications of the resulting two-tier system. It first describes the evaluation environment and metrics, then assesses the weather forecaster and wake-up decider, and finally relates their behavior to measurements of controller- and perception-tier energy consumption.

A. Experimental Setup

We distinguish the environment used for model development from the hardware used for deployment measurements. Wake-up decider training and evaluation were performed on a workstation with 16 GB RAM, an NVIDIA GTX 1070 GPU, and an Intel i7-8750H CPU. Measurements of the deployed forecaster, including inference latency and energy consumption, were collected on the target ESP32.

Target and metrics: Every 5-minute bin is a binary decision instant: a policy *captures* or does not. Throughout the paper, we use the terms *capture* and *shots* interchangeably to denote the act of taking one image in a bin. Relative to the `RPi_sleep` ground-truth variable, a skipped (i.e., not captured) bin with $\texttt{RPi_sleep} = 1$ is a true positive (TP), a captured bin with $\texttt{RPi_sleep} = 1$ is a false negative (FN), a skipped bin with $\texttt{RPi_sleep} = 0$ is a false positive (FP), and a captured bin with $\texttt{RPi_sleep} = 0$ is a true negative (TN). We report:

$$\textit{Accuracy} = \frac{\mathrm{TP}+\mathrm{TN}}{\mathrm{TP}+\mathrm{TN}+\mathrm{FP}+\mathrm{FN}} \qquad \textit{Precision} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FP}}$$

$$\textit{Recall} = \frac{\mathrm{TP}}{\mathrm{TP}+\mathrm{FN}} \qquad \textit{Specificity} = \frac{\mathrm{TN}}{\mathrm{TN}+\mathrm{FP}}$$

$$F_1 = \frac{2 \cdot \textit{Precision} \cdot \textit{Recall}}{\textit{Precision} + \textit{Recall}}$$

alongside two deployment-facing quantities: *Shots* (FN+TN), defined as the number of captured bins, and *Save*, defined as the percentage reduction in captures achieved by a policy relative to the Blanket Check policy, which captures every bin:

$$\textit{Save} = 100 \times \left(1 - \frac{\textit{Shots}}{N}\right)$$

where N is the total number of bins in the evaluation set (so $\textit{Shots} = N$, and hence $\textit{Save} = 0\%$, for the Blanket Check policy). Because absence ($\texttt{RPi_sleep} = 1$) is the overwhelming majority of bins, *Precision* and *Recall* are both large for nearly any policy that mostly skips. They mainly verify that empty bins are correctly skipped, which is a relatively simple case. *Specificity* is the metric that measures whether a policy detects real BMSB activity. It is computed against the rare, presence class and is the one to read when comparing how much real detection capability a policy has, alongside *Save*. Here and below, the positive class is the skip class ($\texttt{RPi_sleep} = 1$); *Specificity* therefore measures retention of BMSB-positive bins. Unless stated otherwise, all classifiers are evaluated at their native 0.5 probability threshold, without an externally imposed capture budget or *Specificity* target.

Blind baselines: Two policies, Blanket Check and Random Check, that use neither weather nor biological information provide non-learned reference points for the comparison. The Blanket Check (BC) captures every bin, by construction reaching 100% *Specificity* (it never misses real activity) at the cost of capturing everything, i.e., 0% *Recall* and 0% *Precision*. BC is also the reference count used for computing *Save*. The Random Check (RC) captures each bin independently with fixed probability p; for $p = 0.5$ and $p = 0.3$ (RC_{50}, RC_{30}), we report the corresponding expected values, which equal the long-run average over independent draws.

Wake-up decider model: Stage 2 (Section III-B) uses a supervised classifier to refine the deterministic biological envelope. In the deployed system, this family is instantiated as a `HistGradientBoostingClassifier` (HistGBT), trained with its library defaults and a fixed random seed on 15 weather- and phenology-derived features (temperature, humidity, and its 3-hour trend, dewpoint, vapor-pressure deficit, degree-day total and daily increment, photoperiod length and its trend, a long-day flag, and sine/cosine encodings of time-of-day and day-of-year). The classifier family was selected empirically. Section V-C evaluates six alternatives using the same features: XGBoost, Random Forest, Extra-Trees, logistic regression, Gaussian naïve Bayes, and a Multilayer Perceptron (MLP) (Tables I–II). Among the seven alternatives, Gaussian naïve Bayes is HistGBT's closest competitor in *Specificity*. Compared with Gaussian naïve Bayes, HistGBT dominates in same-camera protocol across all the metrics. Focusing on

cross-camera, it requires 43.3% fewer captures (3,494 vs. 6,163) while retaining 62.4% rather than 72.3% of BMSB-positive bins, a 9.9-percentage-point reduction. We deploy HistGBT because this operating point substantially reduces RPi activations while retaining a majority of BMSB-positive bins under the cross-camera protocol (Table II, Section V-C).

Evaluation protocols: Two camera stations contain sufficient labeled activity for evaluation (Section IV-A). We therefore use both cross-camera and same-camera protocols, which answer different questions. The *cross-camera* protocol trains on one camera and evaluates on the other, in both directions with the results pooled. It is the primary deployment-oriented protocol because each prediction is generated for a camera that was not used during training. Unless stated otherwise, the wake-up decider results use this protocol. The *same-camera* protocol trains and evaluates the model on the same camera it is trained, using a stratified 5-fold split. It shares camera-specific characteristics between training and test data and is more favorable than the cross-camera protocol; comparing the two helps distinguish cross-site generalization error from the intrinsic difficulty of the classification task.

B. Weather Forecaster Evaluation

Temperature and humidity are forecast by two independently trained GRUs that share the architecture described in Section III-B: a single GRU layer with 64 units receives a 5-dimensional input consisting of the current reading and sine/cosine encodings of time-of-day and day-of-year. A two-layer head then produces all τ future steps in one forward pass. Because the model is not autoregressive, recursive rollout error does not arise. We evaluate $\tau \in \{12, 24, 48, 96, 192, 288\}$ future steps at the native 5-minute resolution. These values correspond to forecast horizons of 1, 2, 4, 8, 16, and 24 h, respectively, and match the deployed single-reading input.

***Accuracy* across horizons and model configurations:** Figure 5 reports Mean Absolute Error (MAE) and the coefficient of determination R^2 for the dedicated per-variable GRU over the full held-out test set, which contains approximately 29,400–29,700 windows per horizon. For temperature, R^2 decreases from 0.971 at 1 h to 0.509 at 16 h. Humidity exhibits a larger degradation over the same range, with R^2 decreasing from 0.951 to 0.418. The results support a variable look-ahead horizon: the controller can trade sensing frequency against forecast uncertainty by selecting τ.

Figure 5 also reports a joint model in which one GRU predicts both variables, rather than using the two independent models adopted in the deployed system. The joint model has more trainable parameters at every horizon, e.g., 22,616 versus 21,644 parameters at the 1-hour horizon, because its input and output cover both variables. Nevertheless, its MAE differs from that of the dedicated model by only a few hundredths to a few tenths at most horizons; at the 16-hour horizon for temperature, the joint model yields 3.482 °C versus 3.581 °C for the dedicated model. The similar short-horizon results indicate that the choice between dedicated and joint models is primarily architectural in this setting. Hence, the deployed

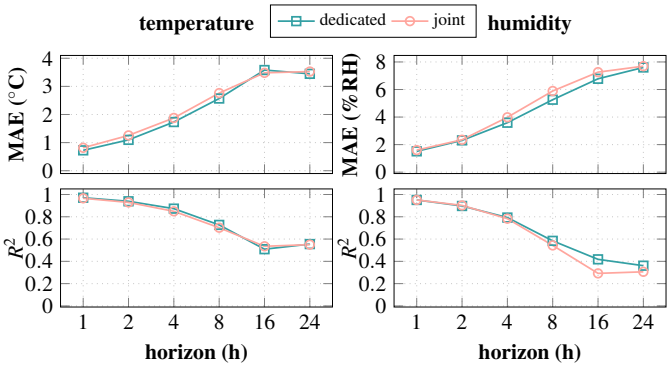


Fig. 5. Forecast MAE (top) and R^2 (bottom) vs. horizon: temperature (left), humidity (right); per-variable vs. joint 2-var GRU.

system retains the dedicated models because they provide a simpler separation of the two forecasting tasks.

Parameters, compute, and quantization: Model size depends on the horizon only through the output head. The recurrent core remains fixed at 64 units, while the parameter count increases from 21,644 at the 1-hour horizon to 48,608 at the 8-hour horizon. The per-inference operation count, measured in multiply-accumulates (MACs), increases from 21,184 to 48,064 over the same range, remaining below 0.05 MMAC (50,000 MACs) for every tested horizon. GFLOPs are not reported because the deployed C kernel and ESP32 benchmark harness instrument MACs rather than floating-point operations.

Weight-only `int8` quantization reduces the flash footprint by 3.76–3.87× across horizons (84.6 kB → 22.5 kB at the 1-hour horizon and 189.9 kB → 49.1 kB at the 8-hour horizon), while activations and accumulators remain in `float32`.

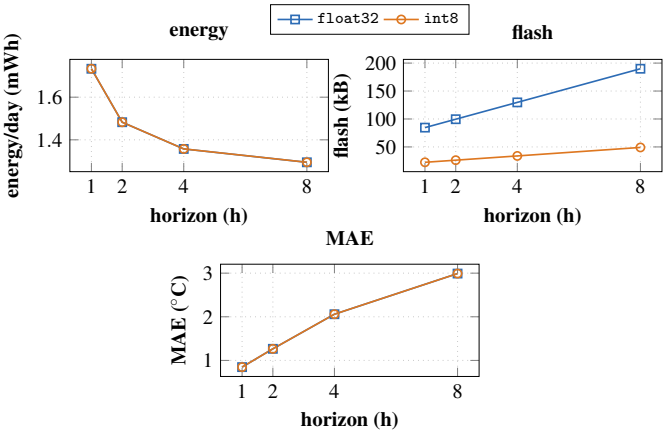


Fig. 6. Effect of weight-only `int8` quantization on the deployed temperature forecaster across horizons $\tau \in \{1, 2, 4, 8\}$ h, the range over which we collected matched on-device energy measurements. The panels report projected daily forecast energy (top left), flash footprint (top right), and MAE (bottom), comparing `float32` and `int8`.

Figure 6 shows all three quantities over $\tau \in \{1, 2, 4, 8\}$ h, the full range for which we have matched on-device measurements; we do not report 16 h or 24 h because at those horizons the per-format energy delta falls below the power meter noise floor and cannot be reliably attributed to quantization. MAE shifts by at most 0.0004 °C between formats at any tested horizon (e.g., both formats yield 0.847 °C at 1 h and 2.990 °C at 8 h when rounded to three decimal places); visually, the

two curves are indistinguishable. Daily forecast energy under the bare-module projection is likewise unaffected, differing by no more than $0.0001\,\mathrm{mWh}$ between formats at every horizon (e.g., $1.7333 \rightarrow 1.7334\,\mathrm{mWh}$ at $1\,\mathrm{h}$ and $1.2946 \rightarrow 1.2947\,\mathrm{mWh}$ at $8\,\mathrm{h}$). On-device latency, measured on the target ESP32 with radios disabled and the CPU at $240\,\mathrm{MHz}$, likewise differs by no more than about $20\,\mu\mathrm{s}$ at every tested horizon ($6{,}603\,\mu\mathrm{s}$ for `float32` versus $6{,}623\,\mu\mathrm{s}$ for `int8` at $1\,\mathrm{h}$; $14{,}154\,\mu\mathrm{s}$ versus $14{,}150\,\mu\mathrm{s}$ at $8\,\mathrm{h}$).

This absence of an energy or latency benefit is expected, not a measurement artifact. The deployed quantization is weight-only: each `int8` weight is dequantized to `float32` before the multiply-accumulate, so the recurrent core executes the same number of floating-point MACs, on the same floating-point unit, regardless of weight format; only the bytes read from flash shrink, not the arithmetic performed. The target board is a plain ESP32 (dual-core Xtensa LX6), which, unlike the ESP32-S3, has no integer/SIMD matrix-multiply acceleration path that a native `int8` kernel could exploit; even a kernel that skipped dequantization would gain little on this core. The dequantization step itself adds a small per-weight overhead that roughly cancels the reduced memory traffic from smaller weights, which is why `int8` is occasionally marginally slower than `float32` (e.g., 6,623 vs. $6{,}603\,\mu\mathrm{s}$ at $1\,\mathrm{h}$) rather than faster. Per-wake energy is further dominated by fixed costs, boot, sensor read, and wake sequencing, common to both formats, which dilutes whatever small compute-level difference remains. On this hardware, weight-only quantization is a memory footprint optimization rather than an energy optimization: it more than triples the flash capacity available for storing forecasting models without measurably affecting accuracy, latency, or energy.

C. Wake-Up Decider Evaluation

The wake-up decider is evaluated in two parts: we first quantify the captures saved by the deterministic biological envelope, then assess the learned refinement under same-camera and cross-camera protocols. We report classification metrics and capture counts to relate detection behavior to energy implications.

Non-learned reference policies: We evaluate four policies without a learned scoring stage: (1) ENV, the deterministic biological envelope of Stage 1 (see Figure 1); (2–3) Random Check RC_{50} and RC_{30}, which capture each bin with probability 0.5 and 0.3, respectively; and (4) BC, which captures every bin. BC is used as reference to compute the *Save* metric. Tables I and II report *Shots*, *Budget*, and *Save* as per-camera annual averages normalized to $105{,}120$ 5-minute instants (i.e., a full year acquisition). ENV and RC apply the same fixed, camera-independent rule to both stations and therefore give identical results under the same-camera and cross-camera protocols; for this reason we report them once, in Table I, rather than also repeating them in Table II. BC is reported in both tables, as the reference point against which *Save* is computed for the corresponding evaluation set.

The learned policies: The proposed policy combines ENV with a learned refinement, which is screened on the smaller, $57{,}022$-bin envelope-restricted set described in Section IV-A. ENV handles clear-cut, biologically implausible steps, whereas the learned refinement reduces wake-ups among the remaining biologically plausible but ambiguous cases. The envelope-restricted dataset is obtained by applying a deterministic filter and requires no model training or cross-station generalization. Its annual per-camera capture budget is 27.1%, corresponding to a 72.9% reduction relative to blanket capture while retaining 99.9% of the manually labeled BMSB-positive bins.

The performance of the same-camera and cross-camera protocols is reported in Tables I and II, respectively.

Same-camera classification: Within the data filtered by the biological envelope of Section III-B, we evaluate seven classifiers: (1) HistGBT, (2) XGBoost, (3) Random Forest, (4) Extra-Trees, (5) logistic regression, (6) Gaussian naïve Bayes, and (7) MLP. Each model is trained and evaluated on the same camera using stratified, shuffled 5-fold cross-validation.

The results are reported in Table I. The learned classifiers produce annual capture budgets between 1.0% and 6.3%. MLP attains the smallest capture budget (1.0%) and the highest *Recall* (0.994), but its *Specificity* is only 48.2%; thus, it misses more than half of the BMSB-positive intervals. Extra-Trees and HistGBT obtain the highest *Specificity* values, 97.4% and 96.8%, respectively. Extra-Trees also obtains the highest *Receiver Operating Characteristic Area Under the Curve (ROC-AUC)* (0.969) and *Precision* (1.000), whereas MLP obtains the highest *Recall* (0.994) and F_1 score (0.995) at a 1.0% capture budget. Random Forest and XGBoost provide intermediate capture budgets (2.3% and 2.1%) with *Specificity* values of 92.8% and 86.2%, respectively, whereas logistic regression and Gaussian naïve Bayes retain more capture opportunities (6.3% and 6.2%) while reaching 93.7% and 91.0% *Specificity*.

TABLE I
SAME-CAMERA CLASSIFICATION AT NATIVE THRESHOLD, INCLUDING NON-LEARNED POLICIES.

Method	Budget (%) ↓	Shots ↓	Save (%) ↑	ROC-AUC ↑	Recall ↑	Prec. ↑	Spec. ↑	F_1 ↑
Extra-Trees	2.8	2,968	97.2	**0.969**	0.979	**1.000**	**0.974**	0.989
Gaussian NB	6.2	6,552	93.8	0.902	0.944	0.999	0.910	0.971
HistGBT	3.1	3,276	96.9	0.967	0.976	**1.000**	0.968	0.988
Logistic	6.3	6,647	93.7	0.919	0.943	0.999	0.937	0.970
MLP	**1.0**	**1,052**	**99.0**	0.958	**0.994**	0.997	0.482	**0.995**
Random Forest	2.3	2,464	97.7	0.967	0.983	0.999	0.928	0.991
XGBoost	2.1	2,233	97.9	0.963	0.985	0.999	0.862	0.992
ENV	27.1	28,511	72.9	–	0.734	1.000	**0.999**	0.847
RC_{50}	50.0	52,560	50.0	–	0.500	0.009	0.500	0.018
RC_{30}	30.0	31,536	70.0	–	0.300	0.009	0.700	0.017
BC	100.0	105,120	0.0	–	0.000	–	**1.000**	–

Cross-camera classification: The same seven classifiers are retrained on one camera and evaluated on the other, with both directions pooled (Table II). Performance varies substantially across models. Relative to the same-camera results, all learned policies lose *Specificity* under cross-camera evaluation, although to different extents. This degradation indicates a dependency on camera position, while the retained *Specificity* values show that the models preserve some ability to retain BMSB-positive intervals at an unseen station. MLP captures 0.6% of the annual schedule and achieves only 5.7% *Specificity*. HistGBT captures 3.3% and achieves 62.4% *Specificity*. Logistic regression and Gaussian naïve Bayes achieve higher mean *Specificity* values, 74.0% and 72.3%, respectively, but do so at capture rates of 8.2% and 5.9%.

These results are relevant to deployment because Stage 2 uses one fixed threshold and does not perform per-station threshold calibration. Consequently, performance under the native threshold is a more relevant selection criterion than performance obtained after a tuning procedure unavailable at deployment. Among the learned policies, logistic regression achieves the highest *Specificity* (0.740) and ties for the highest *Precision* (0.998), at the cost of a much larger capture budget (8.2%) than HistGBT. Learned policies capture fewer images while retaining nonzero detection; reductions must be read jointly with *Specificity*, not as an independent objective.

TABLE II
CROSS-CAMERA CLASSIFICATION AT NATIVE THRESHOLD, INCLUDING THE NON-LEARNED BC REFERENCE.

Method	Budget (%) ↓	Shots ↓	Save (%) ↑	ROC-AUC ↑	Recall ↑	Prec. ↑	Spec. ↑	F_1 ↑
Extra-Trees	2.5	2,600	97.5	0.831	0.979	0.996	0.496	0.987
Gaussian NB	5.9	6,163	94.1	0.829	0.946	**0.998**	0.723	0.971
HistGBT	3.3	3,494	96.7	0.801	0.971	0.997	0.624	0.984
Logistic	8.2	8,648	91.8	0.780	0.923	**0.998**	**0.740**	0.959
MLP	**0.6**	**582**	**99.4**	0.761	**0.995**	0.993	0.057	**0.994**
Random Forest	2.0	2,073	98.0	**0.832**	0.983	0.996	0.443	0.990
XGBoost	1.9	2,043	98.1	0.802	0.984	0.996	0.421	0.990
BC	100.0	105,120	0.0	–	0.000	–	**1.000**	–

The cross-camera reduction in performance with respect to the same-camera protocol is consistent with the spatial structure of the field. BMSB aggregation is local within an orchard: particular trees or canopy zones may exhibit substantially greater activity than neighboring areas, potentially due to aggregation pheromones [37]. Cameras at different positions therefore observe different local behavioral regimes that cannot be fully represented by weather and degree-day features shared across the orchard. The same-camera results characterize performance when camera-specific conditions are shared between training and evaluation. The cross-camera results provide the deployment-relevant estimate because a newly installed station is not represented in the training data. The gap between the two protocols is itself informative: because cross-camera performance is consistently worse rather than merely different, camera-position dependency is a real limitation of the learned policies rather than a diagnostic curiosity, and it cannot be ignored when planning a deployment at a new station.

Projected annual RPi-side energy from *Save*: The two capture policies power the RPi differently: BC boots the board once and idle-holds it continuously, capturing and running an on-board YOLO inference every $5\,\mathrm{min}$, whereas a decider-gated policy boots the RPi only for each wake the decider returns, captures, infers, and switches the board off until the next wake. Energy is therefore *not* proportional to *Shots* alone; it also depends on idle-hold power and per-wake boot cost. We measured $P_{\text{idle}} \approx 2.7$–$3.4\,\mathrm{W}$ (midpoint $3.05\,\mathrm{W}$) idle-hold and $\approx 5.5\,\mathrm{W}$ over a $27\,\mathrm{s}$ boot on our own Raspberry Pi 5 hardware, giving $E_{\text{boot}} \approx 148.5\,\mathrm{J}$. For per-inference energy we reuse the smallest-YOLO measurement from our companion deployment [26], $E_{\text{inf}} = 6.18\,\mathrm{W} \times 0.346\,\mathrm{s} \approx 2.14\,\mathrm{J}$. No official current-draw specification for Camera Module 3 exists in any RPi datasheet, so we bound the capture cost above by the camera connector's documented $600\,\mathrm{mA}/3.3\,\mathrm{V}$ budget over an assumed $1\,\mathrm{s}$ capture pipeline, $E_{\text{capt}} \lesssim 2\,\mathrm{W} \times 1\,\mathrm{s} = 2\,\mathrm{J}$ [39], the one remaining unmeasured term, flagged further in Section VI.

The two stations contributing to *Shots* were deployed in parallel throughout 2023. The BC reference follows the full annual schedule, with $2 \times 105{,}120 = 210{,}240$ 5-minute capture opportunities across both stations, independently of weather-data validity. For the decider-gated policies, we use the annualized per-camera *Shots* reported in Tables I and II; these projections extrapolate decisions observed on valid bins and do not prescribe behavior during missing-sensor intervals. For the idle-hold term, we use $T_{\text{seas}} = 365\,\mathrm{days}$ per station. Total annual energy under each policy is

$$E_{\text{BC}} = 2\left(E_{\text{boot}} + P_{\text{idle}} T_{\text{seas}}\right) + \text{Shots}_{\text{BC}}\left(E_{\text{capt}} + E_{\text{inf}}\right),$$
$$E_{\text{method}} = \text{Shots}_{\text{method}}\left(E_{\text{boot}} + E_{\text{capt}} + E_{\text{inf}}\right),$$

where the factor of 2 accounts for both stations idle-holding through the year under BC, and the projected RPi-side energy-reduction factor is $E_{\text{BC}}/E_{\text{method}}$. With $\text{Shots}_{BC} = 210{,}240$, idle-hold power dominates E_{BC}: over 99% of the $\approx 53.8\,\mathrm{kWh}$ annual total across both stations, versus $\approx 0.24\,\mathrm{kWh}$ from captures, inference, and the single annual boot combined. For these two-station energy estimates, the decider-gated policies use pooled shot counts, i.e., twice the per-camera annual averages reported in the tables. In the same-camera results, MLP requires $2{,}104$ *Shots*, corresponding to a projected RPi-side energy reduction by a factor of ≈ 603 relative to BC. Logistic regression requires $13{,}294$ *Shots*, corresponding to a factor of ≈ 95. In the cross-camera results, HistGBT requires $6{,}989$ *Shots*, corresponding to a factor of ≈ 181. MLP requires only $1{,}164$ *Shots* in the cross-camera protocol, corresponding to a factor of $\approx 1{,}090$, but its 5.7% *Specificity* under the cross-camera protocol shows that this reduction comes with limited detection capability. *Save* and these multipliers should not be interpreted independently of *Specificity* or *Recall*; the HistGBT/Gaussian naïve Bayes model-selection trade-off is detailed in Section V-A.

D. Energy Consumption and Autonomy

This section quantifies the measured cost of the controller tier and compares it with the perception-tier cost that the controller is intended to avoid.

Forecast instead of sense: The WITRN J7-C inline meter used in Section V-B samples the USB $5\,\mathrm{V}$ rail at $1\,\mathrm{Hz}$ with $10\,\mathrm{mA}$ resolution, too coarse to directly resolve a sub-second boot or a $6.6\,\mathrm{ms}$ inference within a single wake cycle. We therefore decompose each duty cycle into its constituent phases, boot, sensor read, inference, and deep sleep, and measure them separately: each phase is held as a steady load long enough for the meter to report a stable current, while phase durations are obtained on-device, short durations from the ESP32 `esp_timer` and wake duration from a boot count over a capture interval of known length. Per-phase energy is current multiplied by duration, and the energy of a full wake cycle is the sum of its phase energies. This composed *bare-module projection* additionally substitutes the datasheet deep-sleep current of the ESP32 chip alone ($10\,\mu\mathrm{A}$) for the measured sleep-phase current, isolating the controller energy budget from the regulator, USB-UART bridge, and other components present on our development board but absent from a purpose-built deployment enclosure; every energy figure below is computed under this composed, bare-module assumption.

We compare two such composed duty cycles that produce the same stream of 5-minute readings. The sense node wakes, reads, and sleeps every $5\,\mathrm{min}$, whereas the forecast node wakes once per horizon τ, performs one reading and one GRU inference (Section V-B), and then returns to sleep. Radios are disabled in both configurations. This comparison is conservative for the sense node because a deployed sensing system would also incur the cost of transmitting the additional readings replaced by forecasts in the forecast configuration.

TABLE III
DAILY ENERGY GAIN BETWEEN FORECAST NODE VS. ALWAYS-SENSING NODE, ACROSS FORECAST HORIZONS.

Horizon (h)	Forecast E_{day} (mWh)	Gain (×)
1	1.733	4.062
2	1.483	4.749
4	1.357	5.187
8	1.295	5.438
16	1.263	**5.573**

Across horizons of 1, 2, 4, 8, and $16\,\mathrm{h}$, the forecast node consumes 4.06–$5.57\times$ less energy per day than the sense node. The gain increases quickly at first and then flattens: from $4.06\times$ at $1\,\mathrm{h}$ to $5.44\times$ at $8\,\mathrm{h}$, and only to $5.57\times$ at $16\,\mathrm{h}$. This range also covers the horizons beyond which forecast accuracy begins to saturate. Although the projected gain is highest at $16\,\mathrm{h}$, forecasts at this horizon are unreliable (Section V-B); the value should therefore be viewed as an energy extrapolation rather than as a reliable operating point. We report the $1\,\mathrm{h}$ figure, $4.062\times$, as the headline gain elsewhere in this paper (Section VII) because it is the shortest, most conservative horizon at which the forecaster is reliable (Section V-B); the full 4.062–$5.573\times$ range is given here for completeness. Table III reports this comparison for each horizon.

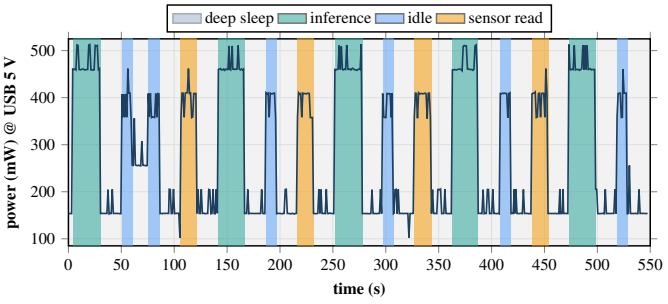


Fig. 7. Measured ESP32 board power across three forecast-node wake cycles (USB $5\,\mathrm{V}$ rail). The unshaded intervals are deep sleep; shaded intervals distinguish inference, idle, and sensor-read phases.

Figure 7 shows three repeated wake cycles. The idle, sensor-read, and inference phases occupy distinct power levels above the common sleep floor; inference is both the shortest and the highest-power phase, consistent with the approximately $90\,\mathrm{mA}$ current used in the comparison.

Composing the two tiers: The per-inference energy of the controller is small relative to the perception-tier capture cost. A complete RPi wake, including its own boot, capture, and inference, requires $E_{\text{boot}}+E_{\text{capt}}+E_{\text{inf}} \approx 152.6\,\mathrm{J}$ under the constants introduced in Section V-C. The measured per-inference delta is 0.5–$1\,\mathrm{mJ}$ above idle, or 0.0003–0.0007% of this $\approx 152.6\,\mathrm{J}$ RPi wake. Thus, the marginal controller overhead is below 0.001% of one RPi wake. A complete forecast wake, including boot, sensor read, and inference, requires 75.7–$83.2\,\mathrm{mJ}$ across horizons, corresponding to 0.050–0.055% of one RPi wake. Although this bound includes all controller-wake phases, it remains small relative to the perception-tier cost. Consequently, modeled RPi-side energy is determined primarily by the number of RPi activations, or equivalently by the *Save* achieved by the wake-up decider (Section V-C). The reported annual savings reach up to 99.0% for same-camera evaluation and 99.4% for cross-camera evaluation. The two mechanisms therefore address complementary energy terms: forecasting reduces the sensing frequency of the controller, while the decider determines how often the multi-joule perception-tier capture is initiated.

VI. Discussion and Outlook

The evaluation above was intentionally conservative wherever a choice was available: it reports the cross-camera protocol, which evaluates a camera unseen during training rather than relying only on the more favorable same-camera protocol; moreover, while idle-hold and boot power are now measured directly on our own RPi hardware, the per-inference energy is still a companion-deployment estimate [26] and the capture term remains a documented bound rather than a measurement. Every ESP32 duty-cycle phase was measured directly on our devkit hardware; the one departure from measurement is the bare-module sleep-current projection underlying the daily-energy comparison (Section V-D), which substitutes the ESP32 datasheet deep-sleep current for our measured value and has not yet been validated on a production board. This section separates the conclusions supported by the evidence from the limitations that still require validation.

What the results support: The central result is the complementary two-stage decider: the deterministic biological envelope exploits calendar, clock, and temperature cues to remove 72.9% of annual captures while retaining 99.9% of manually labeled BMSB-positive bins (Section V-C) at negligible computational cost, while the learned refinement resolves ambiguous cases within the remaining biologically plausible window. Because it consists of three explicit threshold checks on calendar, clock, and temperature, its behavior is auditable and does not depend on a learned model. The learned stage is evaluated even under the protocol faced by a newly deployed station: training uses one camera and evaluation uses another. Even under this setting, it achieves a more favorable capture-reduction–activity-retention trade-off than the blind random-check policies (Table II). On the controller side, the forecaster degrades gracefully as the horizon increases (Section V-B), predicts all τ steps in one forward pass, and quantizes to `int8` with essentially no accuracy loss. When combined with the decider, a complete controller wake for sensing, forecasting, and decision-making costs at most 0.055% of a single RPi wake (Section V-D). The two-tier premise therefore holds even before accounting for the captures avoided by the decider.

Where the evidence runs out: Three limitations qualify this interpretation. First, cross-camera *Specificity* is below same-camera *Specificity* for every method (Tables I–II). Only two

stations contained enough labeled activity for this comparison. The cross-site gap may reflect the locally clustered, pheromone-influenced aggregation described in Section V-C, but the available sites are insufficient to distinguish a biological effect from station-specific variation. The wide variation across classifiers in Table I, where *Specificity* ranges from 48.2% (MLP) to 97.4% (Extra-Trees), also shows that performance under this less demanding protocol is not uniformly stable across model choices.

Second, the contribution of the learned stage is conditional. Stage 2 uses a fixed, globally trained threshold and does not perform per-station calibration. Third, a narrower gap is the on-device vision filter described in Section III-C, which is specified architecturally but not evaluated end-to-end. Its accuracy–energy trade-off remains an open question.

Net assessment: Overall, the central claims hold under the most conservative combination tested for classification: cross-camera scoring, with an energy model that still leans on a companion-deployment inference estimate and a documented capture-cost bound. The controller-overhead claim, at most 0.055% of one RPi wake, follows directly from measured device-tier energies and does not depend on any sleep-current assumption; the larger 4.06–$5.57\times$ daily-energy multiplier is already supported by the bare-module sleep-current projection and will be strengthened further once validated on a production board. The system discards most images while retaining real activity, and the controller cost remains negligible relative to the perception tier it gates. The results are likely to improve further with a production board, per-station threshold calibration, and a directly measured RPi phase model, though none of these refinements is required to support the conclusions above.

Outlook: The remaining gaps suggest concrete next steps. The radio path should be measured end-to-end rather than assumed to be inactive. The bare-module sleep-current projection should be validated on a production board. The remaining bounded terms, per-inference energy and camera-capture current, should be replaced with a phase-resolved measurement of our own detector running on our own hardware. Stage 2 should be extended from one global threshold to per-station calibration or online adaptation, so that a newly deployed camera is not evaluated at the operating point learned at another site. An Reinforcement Learning (RL) policy could further adapt wake-up decisions to orchard zones by using the on-device object detector's outcomes as a reward signal. The dataset should be expanded beyond two cameras and one season to distinguish genuine cross-site generalization from small-sample variation. The on-device vision filter should be evaluated end-to-end.

VII. Conclusions

Fixed-interval acquisition wastes energy because it captures images without considering whether the target phenomenon is likely to be active. This paper addresses that problem with a low-power ESP32 controller that remains in deep sleep, forecasts weather instead of sensing continuously, and wakes an RPi camera only when a biologically informed decider considers BMSB activity likely. The architecture separates *decision* from *perception* across two hardware tiers matched to the cost of each task.

The controller energy measurements were collected on the target hardware, and the system-level projection accounts for the perception-tier cost that the controller avoids. A deterministic biological envelope alone removes 72.9% of annual captures while retaining 99.9% of manually labeled BMSB-positive bins, without a learned model. The gap between same-camera and cross-camera results reveals a spatial component of BMSB activity, offering a clear direction for extending the current non-spatial feature set with location-aware information. Under the deployment-oriented cross-camera protocol, the complete decider achieves a more favorable capture-reduction–activity-retention trade-off than the blind random-check policies (Table II). The forecaster degrades gracefully as the horizon increases and quantizes to `int8` with essentially no accuracy loss. When combined with the decider and measured on real hardware, the controller costs at most 0.055% of a single RPi wake and consumes 4.06–$5.57\times$ less energy than continuous sensing.

These results were obtained under stated assumptions, including cross-camera evaluation, disabled radios, and a companion-deployment RPi inference-energy estimate; the reported 4.06–$5.57\times$ daily-energy gain additionally relies on the bare-module sleep-current projection discussed in Section VI, pending validation on a production board. They support the central claim that a biologically informed controller can avoid most unnecessary captures while retaining real activity, at a decision cost that remains negligible relative to the perception tier it gates.

References

[1] L. Almstedt *et al.*, "A comprehensive pest monitoring system for brown marmorated stink bug," *IEEE Transactions on AgriFood Electronics*, vol. 3, pp. 110–120, 2025.

[2] A. Kargar *et al.*, "Detecting halyomorpha halys using a low-power edge-based monitoring system," *Computers and Electronics in Agriculture*, vol. 221, p. 108935, jun 2024.

[3] M. C. F. Lima *et al.*, "Automatic detection and monitoring of insect pests—a review," *Agriculture*, vol. 10, p. 161, 2020.

[4] M. Bariselli *et al.*, "Distribution and damage caused by halyomorpha halys in italy," *Eppo Bulletin*, vol. 46, pp. 332–334, 2016.

[5] L. Almstedt *et al.*, "Beyond the naked eye: Computer vision for detecting brown marmorated stink bug and its punctures," *IEEE Transactions on AgriFood Electronics*, vol. 3, pp. 6–17, 2025.

[6] A. Kargar *et al.*, "Tiny deep learning model for insect segmentation and counting on resource-constrained devices," *Computers and Electronics in Agriculture*, vol. 236, p. 110378, sep 2025.

[7] L. Palazzetti *et al.*, "The hawk eye scan: Halyomorpha halys detection relying on aerial tele photos and neural networks," *Computers and Electronics in Agriculture*, vol. 226, p. 109365, 2024.

[8] K. Bjerge *et al.*, "Towards edge processing of images from insect camera traps," *Remote Sensing in Ecology and Conservation*, apr 2025.

[9] F. Betti Sorbelli *et al.*, "Wireless iot sensors data collection reward maximization by leveraging multiple energy- and storage-constrained uavs," *Journal of Computer and System Sciences*, vol. 139, p. 103475, feb 2024.

[10] F. Betti Sorbelli *et al.*, "Drone-based bug detection in orchards with nets: A novel orienteering approach," *ACM Trans. Sen. Netw.*, vol. 20, apr 2024.

[11] F. Betti Sorbelli *et al.*, "Yolo-based detection of halyomorpha halys in orchards using rgb cameras and drones," *Computers and Electronics in Agriculture*, vol. 213, p. 108228, 2023.

[12] L. Palazzetti *et al.*, "A comprehensive dataset of halyomorpha halys for automated detection and monitoring in orchards," *Scientific Data*, vol. 13, July 2026.

[13] L. Palazzetti *et al.*, "Let me sleep! a machine learning approach for iot wake-up mechanism for insect detection," in *2025 IEEE International Workshop on Metrology for Agriculture and Forestry (MetroAgriFor)*. IEEE, Oct 2025, p. 122–127.
[14] S. Benhamaid *et al.*, "Recent advances in energy management for Green-IoT: An up-to-date and comprehensive survey," *Journal of Network and Computer Applications*, vol. 198, p. 103257, 2022.
[15] R. Piyare *et al.*, "Ultra low power wake-up radios: A hardware and networking survey," *IEEE Communications Surveys & Tutorials*, vol. 19, pp. 2117–2157, 2017.
[16] K. S. Adu-Manu *et al.*, "Energy-harvesting wireless sensor networks (eh-wsns): A review," *ACM Transactions on Sensor Networks*, vol. 14, 2018.
[17] D. E. Ruiz-Guirola *et al.*, "Energy management and wake-up for IoT networks powered by energy harvesting," *IEEE Internet of Things Journal*, vol. 13, pp. 18 574–18 591, 2026.
[18] I. Miro-Panades *et al.*, "Samurai: A versatile iot node with event-driven wake-up and embedded ml acceleration," *IEEE Journal of Solid-State Circuits*, vol. 58, pp. 1782–1797, 2023.
[19] S.-W. Yun *et al.*, "Wake-up security: Effective security improvement mechanism for low power internet of things," *Intelligent Automation & Soft Computing*, vol. 37, p. 2897–2917, 2023.
[20] Z. Wang *et al.*, "Lightlog: A lightweight temporal convolutional network for log anomaly detection on the edge," *Computer Networks*, vol. 203, p. 108616, feb 2022.
[21] T. Qin *et al.*, "Imlads: Intelligent maintenance and lightweight anomaly detection system for internet of things," *Sensors*, vol. 19, p. 958, feb 2019.
[22] X. Zhou *et al.*, "Reconstructed graph neural network with knowledge distillation for lightweight anomaly detection," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 35, pp. 11 817–11 828, 2024.
[23] L. Almstedt *et al.*, "Technological innovations in agriculture for scouting halyomorpha halys in orchards," in *2023 19th International Conference on Distributed Computing in Smart Systems and the Internet of Things (DCOSS-IoT)*, 2023, pp. 702–709.
[24] F. Betti Sorbelli *et al.*, "Preliminary results for halyomorpha halys monitoring relying on a custom dataset," in *2023 IEEE International Workshop on Metrology for Agriculture and Forestry (MetroAgriFor)*, 2023, pp. 363–368.
[25] M. A. Ali *et al.*, "AI-enabled IoT-based pest prevention and controlling system using sound analytics in large agricultural field," *Computers and Electronics in Agriculture*, vol. 220, p. 108844, 2024.
[26] L. Palazzetti *et al.*, "Antpi: A raspberry pi based edge–cloud system for real-time ant species detection using yolo," *Ecological Informatics*, vol. 91, p. 103383, 2025.
[27] V. Rajapakse *et al.*, "Intelligence at the extreme edge: A survey on reformable TinyML," *ACM Computing Surveys*, vol. 55, pp. 1–30, 2023.
[28] S. Heydari *et al.*, "Tiny machine learning and on-device inference: A survey of applications, challenges, and future directions," *Sensors*, vol. 25, p. 3191, 2025.
[29] R. David *et al.*, "Tensorflow lite micro: Embedded machine learning for tinyml systems," in *Proceedings of Machine Learning and Systems*, A. Smola *et al.*, Eds., vol. 3, 2021, pp. 800–811.
[30] A. Kusupati *et al.*, "Fastgrnn: a fast, accurate, stable and tiny kilobyte sized gated recurrent neural network," in *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, ser. NIPS'18. Red Hook, NY, USA: Curran Associates Inc., 2018, p. 9031–9042.
[31] M. El Adoui *et al.*, "Constrained tiny machine learning for predicting gas concentration with I4.0 low-cost sensors," *ACM Transactions on Embedded Computing Systems*, vol. 23, pp. 1–23, 2024.
[32] N. El-Amarty *et al.*, "Accurate short-term solar irradiance forecasting with tinyml on edge device," in *2024 International Conference on Circuit, Systems and Communication (ICCSC)*, 2024, pp. 1–6.
[33] M. Rot *et al.*, "Biological parameters, phenology and temperature requirements of halyomorpha halys (hemiptera: Pentatomidae) in the sub-mediterranean climate of western slovenia," *Insects*, vol. 13, p. 956, 2022.
[34] G. L. Baskerville *et al.*, "Rapid estimation of heat accumulation from maximum and minimum temperatures," *Ecology*, vol. 50, pp. 514–517, 1969.
[35] A. L. Nielsen *et al.*, "Developmental rate estimation and life table analysis for halyomorpha halys (hemiptera: Pentatomidae)," *Environmental Entomology*, vol. 37, pp. 348–355, 2008.
[36] D. L. Musolin *et al.*, "Photoperiodic and temperature control of nymphal growth and adult diapause induction in the invasive caucasian population of the brown marmorated stink bug, halyomorpha halys," *Journal of Pest Science*, vol. 92, p. 621–631, jan 2019.
[37] A. L. Nielsen *et al.*, "Seasonal phenology and monitoring of the non-native halyomorpha halys (hemiptera: Pentatomidae) in soybean," *Environmental Entomology*, vol. 40, pp. 231–238, 04 2011.
[38] R. F. Tate, "Correlation between a discrete and a continuous variable. point-biserial correlation," *Annals of Mathematical Statistics*, vol. 25, pp. 603–607, 1954.
[39] Raspberry Pi Forums, "Power consumption of raspberry pi camera module 2 & 3," https://forums.raspberrypi.com/viewtopic.php?t=347469, 2023, raspberry Pi Foundation engineer confirms all Raspberry Pi camera modules operate within the camera connector's 600 mA / 3.3 V ($\approx$ 2 W) budget; no isolated current-draw figure is published for Camera Module 3.



Francesco Betti Sorbelli He received the B.S. and M.S. degrees in Computer Science from the Univ. of Perugia, Italy, in 2007 and 2010, respectively, and the Ph.D. degree in Computer Science from the Univ. of Florence, Italy, in 2018. He was a post-doctoral researcher at Missouri S&T, USA, in 2020, and at the Univ. of Perugia in 2018 and 2021–2022. Since 2025, he has been an Associate Professor at the Univ. of Perugia. His research interests include algorithm design, combinatorial optimization, and unmanned vehicles.



Enrico Casella He received the B.S. and M.S. degrees in Computer Engineering from University of Palermo in 2015 and 2018, respectively, and the Ph.D. degree in Computer Science from University of Kentucky in 2023. He is currently an Assistant Professor with the Department of Animal Science, at Pennsylvania State University, where he directs the Cyber-Physical animal Systems (CAS) Laboratory. His research interests include artificial intelligence and computer vision for livestock systems. He is a Member of the IEEE.



Lorenzo Palazzetti He received the Ph.D. degree with distinction in Computer Science from the Univ. of Florence, Italy. He was a postdoctoral researcher at the Univ. of Perugia, working on IoT, computer vision, and BVLoS autonomous drone systems. He is currently a postdoctoral researcher at Pennsylvania State University, USA, where he conducts AI- and sensor-based research for smart agricultural monitoring. His research interests include autonomous systems, computer vision, IoT, and combinatorial optimization.



Cristina M. Pinotti She received the M.S. degree cum laude in Computer Science from the Univ. of Pisa, Italy, in 1986. From 1987 to 1999, she was a Researcher with the National Research Council in Pisa. From 2000 to 2003, she was an Associate Professor at the Univ. of Trento. Since 2004, she has been a Full Professor at the Univ. of Perugia. Her research interests include the design and analysis of algorithms, combinatorial optimization, emerging technologies, wireless sensor networks, and communication networks.